

ROSIT

《从小乌龟到机器人系统：ROS1 实践入门》

以 turtlesim 为主线，理解机器人软件的组织方式

前言

为什么用 turtlesim 写一本 ROS1 教材

很多人学习 ROS 时，第一感觉是“命令很多、概念很多、文件很多”。

节点、话题、服务、动作、参数、launch、TF、rosviz、RViz、Gazebo、导航、机械臂……这些概念如果一次性摆出来，学生很容易觉得 ROS 是一堆工具的集合，而不是一个可以理解的系统。

本书选择从 turtlesim 出发。

turtlesim 很小，但它不是玩具。

它提供了一个足够简单、足够稳定、足够可观察的机器人实验世界。在这个世界里，乌龟有位置、有速度、有朝向，可以接收控制指令，可以被生成、删除、重命名，可以通过 topic 发布状态，也可以通过 service 接受请求。

这意味着，我们完全可以把 turtlesim 看成一个二维机器人实验场。

本书的目标不是让学生“记住 ROS 命令”，而是让学生理解：

- 一个机器人系统怎样被拆成多个节点；
- 节点之间怎样交换数据；
- 程序怎样被组织成 package；
- 多个节点怎样被 launch 统一启动；
- 系统出错时怎样调试；
- 一个简单控制程序怎样逐步演化为完整机器人应用；

- ROS1 的系统思想怎样迁移到真实机器人和 ROS2。
-

本书使用方式

本书每一章都围绕一个具体任务展开。

每章基本结构如下：

1. 本章任务
2. 先做出最小可运行程序
3. 观察运行现象
4. 解释背后的 ROS 概念
5. 改进程序结构
6. 常见错误与调试方法
7. 本章练习
8. 小结：本章到底学会了什么

本书建议使用环境：

- Ubuntu 20.04
 - ROS Noetic
 - Python 3
 - catkin 工作空间
 - turtlesim
 - rqt_graph
 - rqt_plot
 - rosbag
 - RViz
 - tf / tf2
-

第一篇 小乌龟醒来：进入 ROS 世界

第 1 章 先让小乌龟出现在屏幕上

1.1 机器人软件为什么需要 ROS

从一个简单问题开始：

如果一个程序负责控制速度，另一个程序负责显示位置，它们怎样合作？

1.2 启动第一个 ROS 世界

任务：启动 roscore 和 turtlesim。

核心命令：

Plain Text

```
1 roscore
2 rosrun turtlesim turtlesim_node
```

本节重点不是命令本身，而是让学生看到：

- ROS 系统不是一个单独程序；
- 一个 ROS 系统通常由多个进程组成；
- roscore 负责让这些进程能够互相发现。

1.3 小乌龟为什么能被键盘控制

任务：使用键盘控制小乌龟运动。

Plain Text

```
1 rosrun turtlesim turtle_teleop_key
```

引出第一个关键问题：

键盘程序和乌龟程序并没有写在一起，为什么按键能让乌龟动？

1.4 ROS 的最小系统图

用图解释：

Plain Text

```
1 keyboard node ---> /turtle1/cmd_vel ---> turtlesim node
```

1.5 本章小结

本章只要求学生理解三句话：

- ROS 系统由节点组成；
- 节点之间通过通信连接；
- turtlesim 是我们后续所有实验的二维机器人世界。

1.6 本章练习

1. 启动 turtlesim。
2. 启动键盘控制节点。
3. 使用 `rostopic list` 查看当前节点。
4. 使用 `rostopic list` 查看当前话题。
5. 尝试关闭键盘节点，观察小乌龟是否还能运动。

第 2 章 第一个节点：让程序拥有 ROS 身份

2.1 普通 Python 程序和 ROS 节点有什么区别

从一个普通 Python 文件开始：

Plain Text

```
1 print("hello turtle")
```

然后提出问题：

这个程序只是自己运行，怎样才能加入 ROS 系统？

2.2 写一个 hello_turtle 节点

任务：创建第一个 ROS Python 节点。

Plain Text

```
1 import rospy
2
3 rospy.init_node("hello_turtle")
4 rospy.loginfo("Hello, turtlesim!")
5 rospy.spin()
```

2.3 节点名字为什么重要

讲解：

- 节点必须有名字；
- ROS 通过名字区分不同节点；
- 同名节点不能同时正常存在。

2.4 用 rosnode 观察节点

实验命令：

Plain Text

```
1 rosnode list
2 rosnode info /hello_turtle
3 rosnode kill /hello_turtle
```

2.5 本章练习

1. 创建一个名为 `my_first_node` 的节点。
2. 让节点每秒输出一次 `"turtle is alive"`。
3. 使用 `rospy info` 查看节点信息。
4. 尝试同时启动两个同名节点，观察现象。
5. 使用匿名节点解决重名问题。

第 3 章 速度控制：让小乌龟按我们的代码运动

3.1 键盘控制不是我们的最终目标

本章任务：

不使用键盘，让程序控制小乌龟运动。

3.2 认识 `/turtle1/cmd_vel`

讲解：

- `/turtle1/cmd_vel` 是速度命令话题；
- 消息类型是 `geometry_msgs/Twist`；
- `linear.x` 控制前进速度；
- `angular.z` 控制旋转速度。

3.3 写一个直行控制节点

任务：让小乌龟持续向前运动。

3.4 写一个转圈控制节点

任务：让小乌龟一边前进一边旋转，形成圆形轨迹。

3.5 为什么乌龟会撞墙

观察现象：

- 乌龟一直前进会撞到边界；
- 控制节点只管发速度，不知道乌龟在哪里。

引出下一章：

如果想让乌龟更聪明，程序必须知道乌龟的位置。

3.6 本章练习

1. 让小乌龟以固定速度直行。

2. 让小乌龟原地旋转。
 3. 让小乌龟画圆。
 4. 修改线速度和角速度，观察圆半径变化。
 5. 思考：控制程序为什么不知道乌龟是否撞墙？
-

第二篇 小乌龟开始说话：Topic 与 Message

第 4 章 位置反馈：让程序知道乌龟在哪里

4.1 只发速度是不够的

本章任务：

读取小乌龟当前位置，并实时打印出来。

4.2 认识 `/turtle1/pose`

讲解：

- `/turtle1/pose` 是乌龟状态话题；
- 消息类型是 `turtlesim/Pose`；
- 包含 `x`、`y`、`theta`、`linear_velocity`、`angular_velocity`。

4.3 写一个位置订阅节点

任务：订阅 `/turtle1/pose` 并打印位置。

4.4 callback 是怎么工作的

重点解释：

回调函数不是我们主动调用的，而是消息到来时 ROS 自动调用的。

4.5 用 `rqt_plot` 画出位置曲线

任务：观察 `x`、`y`、`theta` 随时间变化。

4.6 本章练习

1. 打印小乌龟的 x 、 y 坐标。
 2. 打印小乌龟朝向 θ 。
 3. 判断乌龟是否靠近墙壁。
 4. 使用 `rqt_plot` 观察 x 坐标变化。
 5. 编写程序，当乌龟靠近边界时输出警告。
-

第 5 章 闭环控制：让小乌龟自动走向目标点

5.1 从“盲目运动”到“根据位置运动”

本章任务：

给定一个目标点，让小乌龟自动移动过去。

5.2 目标点与当前位置

需要计算：

- 当前点到目标点的距离；
- 当前朝向与目标方向的夹角；
- 应该前进还是转向。

5.3 最简单的点到点控制器

控制策略：

- 如果角度误差大，先转向；
- 如果角度误差小，向前走；
- 如果距离足够近，停止。

5.4 为什么会抖动

观察现象：

- 接近目标点时左右摇摆；

- 速度过大时会冲过目标点；
- 角度控制和距离控制相互影响。

5.5 引入比例控制思想

用简单方式讲 P 控制：

Plain Text

1 控制量 = 系数 × 误差

5.6 本章练习

1. 让乌龟移动到点 `(5.5, 5.5)`。
2. 让乌龟移动到用户输入的目标点。
3. 比较不同比例系数下的运动效果。
4. 记录一次“冲过目标点”的现象并分析原因。
5. 修改程序，使乌龟到达目标点后停止。

第 6 章 Message：通信数据不是随便写的

6.1 为什么要关心消息类型

本章任务：

设计一个新的消息，描述乌龟任务状态。

6.2 查看已有消息

实验：

Plain Text

```
1 rosmmsg show turtlesim/Pose
2 rosmmsg show geometry_msgs/Twist
```

6.3 自定义 TurtleStatus 消息

设计消息：

Plain Text

```
1 string name
2 float32 x
3 float32 y
4 float32 distance_to_goal
5 bool arrived
```

6.4 生成并使用自定义消息

讲解：

- msg 文件放在哪里；
- package.xml 如何声明依赖；
- CMakeLists.txt 如何配置；
- catkin_make 后如何导入消息。

6.5 消息设计原则

- 字段要少；
- 含义要清楚；
- 单位要明确；
- 不要把无关内容塞进一个消息。

6.6 本章练习

1. 设计一个 `TurtleCommand.msg`。
 2. 设计一个 `TurtleStatus.msg`。
 3. 发布乌龟当前状态。
 4. 订阅状态并判断是否到达目标点。
 5. 思考：什么时候应该使用已有消息，什么时候才需要自定义消息？
-

第三篇 小乌龟接受任务：Service、Action 与参数

第 7 章 Service：让小乌龟接受一次性请求

7.1 Topic 不适合所有问题

Topic 适合连续数据流。

但有些事情只需要请求一次，例如：

- 生成一只新乌龟；
- 清除轨迹；
- 瞬移到某个位置；
- 查询当前任务状态。

7.2 turtlesim 中已有的 Service

实验：

Plain Text

```
1 rosservice list
2 rosservice info /spawn
3 rosservice info /clear
4 rosservice info /turtle1/teleport_absolute
```

7.3 生成第二只乌龟

任务：调用 `/spawn` 生成新乌龟。

7.4 让乌龟瞬移

任务：调用 `/turtle1/teleport_absolute`。

7.5 写一个自己的 Service

设计服务：

Plain Text

```
1 float32 x
2 float32 y
3 ---
4 bool success
5 string message
```

任务：请求乌龟移动到指定点。

7.6 本章练习

1. 用命令行生成一只新乌龟。
 2. 用 Python 调用 `/spawn`。
 3. 用 Python 调用 `/clear` 清除轨迹。
 4. 设计一个服务，让乌龟返回当前是否靠近墙壁。
 5. 思考：为什么不能用 service 控制乌龟持续运动？
-

第 8 章 Action：让小乌龟执行可取消的长任务

8.1 Service 的不足

如果让乌龟从当前位置走到目标点，这个过程可能持续几秒。

这时我们不仅想知道最后结果，还想知道：

- 现在走到哪里了；
- 还剩多远；
- 是否可以取消任务。

这就是 Action 的用武之地。

8.2 一个移动任务包含什么

Action 的三部分：

- Goal：目标点；

- Feedback: 当前进度;
- Result: 最终结果。

8.3 设计 MoveToPoint.action

Plain Text

```
1 float32 target_x
2 float32 target_y
3 ---
4 bool success
5 string result_message
6 ---
7 float32 current_x
8 float32 current_y
9 float32 distance_remaining
```

8.4 编写 Action Server

任务: 让小乌龟执行移动任务, 并持续反馈剩余距离。

8.5 编写 Action Client

任务: 发送目标点, 并显示移动过程。

8.6 任务取消

任务: 当乌龟走到一半时取消目标。

8.7 本章练习

1. 让乌龟移动到指定点并反馈进度。
 2. 给移动任务设置超时时间。
 3. 让用户输入目标点。
 4. 实现任务取消。
 5. 实现多个目标点顺序执行。
-

第 9 章 Parameter：给小乌龟设置性格

9.1 不要把速度写死在代码里

本章任务：

让小乌龟的最大速度、转向速度、停止距离都可以通过参数调整。

9.2 使用 rosparam

实验：

Plain Text

```
1 rosparam set /turtle_controller/max_speed 1.0
2 rosparam get /turtle_controller/max_speed
```

9.3 私有参数

讲解：

Plain Text

```
1 max_speed = rospy.get_param("~max_speed", 1.0)
```

9.4 用 YAML 管理参数

创建配置文件：

Plain Text

```
1 max_speed: 1.0
2 max_angular_speed: 2.0
3 goal_tolerance: 0.1
```

9.5 不同乌龟，不同性格

任务：

- turtle1 速度快；
- turtle2 转向慢；
- turtle3 靠近目标时更谨慎。

引出 namespace parameter。

9.6 本章练习

1. 把目标点控制程序中的速度改为参数。
 2. 用 YAML 文件配置控制参数。
 3. 为两只乌龟设置不同速度。
 4. 修改停止距离，观察到达目标点的行为变化。
 5. 思考：哪些内容适合写成参数，哪些内容不适合？
-

第四篇 小乌龟变成工程：Package、Launch 与系统组织

第 10 章 Package：把零散程序整理成项目

10.1 为什么不能一直写零散脚本

当程序越来越多时，必须回答：

- 脚本放在哪里？
- 消息放在哪里？
- 服务放在哪里？
- 配置文件放在哪里？
- 启动文件放在哪里？

10.2 创建 turtle_robot 包

任务：

Plain Text

```
1 catkin_create_pkg turtle_robot rospy std_msgs geometry_msgs turtlesim
```

10.3 推荐目录结构

Plain Text

```
1 turtle_robot/  
2   scripts/  
3   launch/  
4   msg/  
5   srv/  
6   action/  
7   config/  
8   rviz/  
9   bag/  
10  README.md
```

10.4 package.xml 与 CMakeLists.txt

讲解：

- 依赖声明；
- 消息生成；
- 服务生成；
- action 生成；
- Python 脚本安装。

10.5 本章练习

1. 创建 `turtle_robot` 包。
2. 将已有控制程序放入 `scripts`。
3. 将参数文件放入 `config`。
4. 将启动文件放入 `launch`。
5. 编写 README 说明包的功能。

第 11 章 Launch：一键启动小乌龟系统

11.1 手动启动的痛苦

一个完整实验可能需要多个终端：

- roscore;
- turtlesim_node;
- controller;
- pose_listener;
- action_server;
- rqt_graph。

本章目标：

用一个 launch 文件启动整个系统。

11.2 第一个 launch 文件

Plain Text

```
1 <launch>
2   <node pkg="turtlesim" type="turtlesim_node" name="turtlesim"/>
3 </launch>
```

11.3 启动控制节点

把 turtlesim 和控制节点写进同一个 launch。

11.4 加载参数文件

Plain Text

```
1 <rosparam file="$(find turtle_robot)/config/controller.yaml" command="load"/>
```

11.5 使用 remap 改变话题连接

任务：

不改代码，让控制节点控制不同乌龟。

11.6 include：拆分大型启动文件

推荐拆分：

Plain Text

```
1 bringup.launch
2 controller.launch
3 multi_turtle.launch
4 debug.launch
```

11.7 本章练习

1. 写一个 launch 文件启动 turtlesim。
 2. 写一个 launch 文件启动 turtlesim 和控制节点。
 3. 使用 launch 加载 YAML 参数。
 4. 用 remap 控制 turtle2。
 5. 将系统拆分成多个 launch 文件。
-

第 12 章 Debug：小乌龟不动时怎么办

12.1 机器人系统一定会出错

本章不是教新功能，而是教学生排错。

12.2 常用调试工具

- rosnode
- rostopic
- rosservice

- rosparam
- rqt_graph
- rqt_plot
- roswtf

12.3 小乌龟不动的十种原因

案例：

1. roscore 没启动；
2. turtlesim 没启动；
3. 控制节点没启动；
4. 话题名写错；
5. 消息类型不匹配；
6. 没有持续发布速度；
7. 发布频率太低；
8. 速度值为 0；
9. 节点异常退出；
10. 控制的是 turtle2，但系统里只有 turtle1。

12.4 一套固定排查流程

推荐顺序：

Plain Text

1 看节点 -> 看话题 -> 看消息类型 -> 看消息内容 -> 看发布频率 -> 看参数 -> 看日志

12.5 本章练习

1. 故意写错话题名，观察现象。
2. 故意不启动 turtlesim，观察控制节点表现。
3. 使用 rqt_graph 找出通信断点。
4. 使用 rostopic echo 检查速度命令。
5. 编写一份“小乌龟不动排查表”。

第五篇 多只小乌龟：命名空间、协同与坐标关系

第 13 章 生成多只乌龟：从单机器人到多机器人

13.1 一只乌龟不够用了

本章任务：

生成多只乌龟，并分别控制它们。

13.2 使用 `/spawn`

任务：

- 生成 turtle2；
- 生成 turtle3；
- 指定初始位置；
- 指定初始朝向。

13.3 每只乌龟都有自己的话题

观察：

Plain Text

```
1 /turtle1/cmd_vel
2 /turtle1/pose
3 /turtle2/cmd_vel
4 /turtle2/pose
```

13.4 命名空间的直观理解

解释：

命名空间就像给每只乌龟分配一个房间。房间里的 `cmd_vel` 不会和别的房间混淆。

13.5 删除乌龟

使用 `/kill` 服务删除指定乌龟。

13.6 本章练习

1. 生成三只乌龟。
 2. 分别控制三只乌龟。
 3. 编写程序自动生成 N 只乌龟。
 4. 删除指定乌龟。
 5. 思考：多机器人系统为什么必须重视命名空间？
-

第 14 章 追踪任务：让一只乌龟追另一只乌龟

14.1 从“移动到固定点”到“移动到移动目标”

本章任务：

`turtle2` 自动追踪 `turtle1`。

14.2 订阅两只乌龟的位置

需要同时订阅：

- `/turtle1/pose`
- `/turtle2/pose`

14.3 计算相对位置

计算：

- 两只乌龟之间的距离；
- `turtle2` 到 `turtle1` 的方向；
- `turtle2` 当前朝向与目标方向的误差。

14.4 追踪控制器

控制策略：

- 距离远：前进；
- 方向偏差大：转向；
- 距离足够近：停止。

14.5 追踪中的问题

观察：

- 追踪会有滞后；
- 速度太大容易绕圈；
- 目标突然转向会导致追踪失败。

14.6 本章练习

1. 实现 turtle2 追踪 turtle1。
 2. 调整比例系数，比较追踪效果。
 3. 让 turtle1 随机运动，观察 turtle2 反应。
 4. 让 turtle3 追踪 turtle2，形成链式追踪。
 5. 思考：真实机器人追踪目标时还会遇到哪些问题？
-

第 15 章 编队任务：让多只乌龟排成队

15.1 多机器人协同的第一个例子

本章任务：

turtle2 和 turtle3 跟随 turtle1，并保持指定队形。

15.2 队形不是点，而是相对关系

设计队形：

- 直线队形；
- 三角队形；
- 横向队形；
- 环形队形。

15.3 领导者与跟随者

讲解：

- leader;
- follower;
- formation offset;
- relative pose。

15.4 直线编队实现

任务：

- turtle1 作为 leader;
- turtle2 保持在 turtle1 后方;
- turtle3 保持在 turtle2 后方。

15.5 编队切换

任务：

运行中从直线队形切换到三角队形。

引出参数和服务结合使用。

15.6 本章练习

1. 实现两只乌龟跟随一只乌龟。
2. 实现三角队形。
3. 实现队形切换服务。
4. 记录不同参数下的编队稳定性。
5. 思考：真实多机器人编队还需要哪些信息？

第 16 章 TF 思想：乌龟世界里的坐标关系

16.1 为什么要讲 TF

在 turtlesim 中，每只乌龟的位置都是世界坐标。

但真实机器人中，经常会有多个坐标系：

- 世界坐标；
- 机器人本体坐标；
- 传感器坐标；
- 目标坐标。

本章用乌龟世界提前建立 TF 思想。

16.2 世界坐标和乌龟自身坐标

解释：

- 世界坐标下的位置；
- 乌龟自身前方；
- 乌龟左侧；
- 乌龟右侧。

16.3 将目标点转换到乌龟自身坐标系

任务：

判断目标点在乌龟的前方、后方、左侧还是右侧。

16.4 用 TF 表示多只乌龟关系

任务：

- 发布 `/world` 到 `/turtle1`；
- 发布 `/world` 到 `/turtle2`；
- 查询 `/turtle1` 到 `/turtle2` 的相对关系。

16.5 TF 在真实机器人中的对应

对应关系：

Plain Text

```
1 world      -> map
2 turtle     -> base_link
3 target     -> goal
4 other turtle -> another robot
```

16.6 本章练习

1. 判断目标点在乌龟左边还是右边。
2. 发布 turtle1 的 TF。
3. 发布 turtle2 的 TF。
4. 查询 turtle1 到 turtle2 的相对变换。
5. 用 TF 改写追踪控制程序。

第六篇 小乌龟的数据世界：rosvag、可视化与实验复现

第 17 章 rosvag：把小乌龟的运动录下来

17.1 为什么要记录数据

机器人实验中，很多问题发生一次就很难再现。

rosvag 的作用是：

把一次运行过程保存下来，之后可以反复回放和分析。

17.2 录制乌龟运动

任务：

Plain Text

```
1 rosvag record /turtle1/cmd_vel /turtle1/pose
```

17.3 回放乌龟运动数据

任务：

Plain Text

```
1 rosbag play xxx.bag
```

17.4 离线分析轨迹

分析：

- 轨迹是否平滑；
- 是否接近目标点；
- 速度变化是否合理；
- 是否存在抖动。

17.5 本章练习

1. 录制一次圆形轨迹。
2. 录制一次点到点运动。
3. 回放数据并观察话题。
4. 比较不同控制参数下的 bag 数据。
5. 写一份实验记录报告。

第 18 章 可视化：把系统内部状态画出来

18.1 看见数据，才能理解系统

本章任务：

不只看乌龟运动，还要看速度、误差、距离、状态。

18.2 使用 rqt_plot

可视化：

- x 坐标；
- y 坐标；
- theta；
- 距离误差；
- 角度误差。

18.3 发布调试话题

设计调试消息：

Plain Text

```
1 float32 distance_error
2 float32 angle_error
3 float32 linear_cmd
4 float32 angular_cmd
```

18.4 用图分析控制效果

观察：

- 参数太大时是否振荡；
- 参数太小时是否反应迟钝；
- 到达目标点时误差是否收敛。

18.5 本章练习

1. 发布距离误差话题。
2. 发布角度误差话题。
3. 用 `rqt_plot` 绘制误差变化。
4. 对比三组控制参数。
5. 根据图像解释控制效果。

第 19 章 实验复现：让小乌龟行为可以被重复验证

19.1 为什么课程实验需要可复现

学生做实验时，经常出现：

- 老师这里能跑，学生那里不能跑；
- 今天能跑，明天不能跑；
- 一个参数改了，自己也不知道影响了哪里。

本章目标：

把小乌龟实验变成可重复、可记录、可比较的工程实验。

19.2 固定初始条件

使用 service 设置乌龟初始位置和朝向。

19.3 固定参数

使用 YAML 保存实验参数。

19.4 固定启动方式

使用 launch 统一启动实验。

19.5 固定记录方式

使用 rosbag 记录关键话题。

19.6 实验报告模板

报告包含：

- 实验目标；
- 系统结构图；
- 节点列表；
- 话题列表；
- 参数配置；
- 运行截图；
- 数据曲线；
- 问题分析；

- 改进方案。

19.7 本章练习

1. 设计一个可复现实验：乌龟从 A 点走到 B 点。
 2. 固定初始位置。
 3. 固定控制参数。
 4. 录制 rosbag。
 5. 写一份实验报告。
-

第七篇 小乌龟有了行为：状态机、任务系统与智能体

第 20 章 状态机：让小乌龟不只是执行一条命令

20.1 一个复杂任务不能只靠 if-else

本章任务：

让小乌龟完成“出发—巡逻—返回—停止”的流程。

20.2 什么是状态

设计状态：

Plain Text

```
1 IDLE
2 GO_TO_START
3 PATROL
4 RETURN_HOME
5 STOP
6 ERROR
```

20.3 状态如何切换

讲解：

- 当前状态；
- 触发条件；
- 下一个状态；
- 每个状态对应的动作。

20.4 实现一个巡逻状态机

任务：

- 起点；
- 巡逻点 1；
- 巡逻点 2；
- 巡逻点 3；
- 返回起点。

20.5 异常状态

处理：

- 超时；
- 到达失败；
- 参数错误；
- 乌龟靠近边界。

20.6 本章练习

1. 实现两点往返巡逻。
2. 实现三点巡逻。
3. 加入超时判断。
4. 加入 ERROR 状态。
5. 用状态机改写 Action 任务队列。

第 21 章 行为切换：巡逻、追踪、逃跑与回家

21.1 同一只乌龟可以有多种行为

本章任务：

让乌龟在不同模式之间切换。

行为包括：

- 巡逻；
- 追踪；
- 逃跑；
- 回家；
- 停止。

21.2 行为接口设计

每个行为都回答三个问题：

- 当前目标是什么？
- 应该发布什么速度？
- 什么时候结束或切换？

21.3 追踪行为

turtle2 追踪 turtle1。

21.4 逃跑行为

turtle2 远离 turtle1。

21.5 回家行为

乌龟回到预设 home 点。

21.6 行为切换服务

设计 service：

Plain Text

```
1 string behavior_name
2 ---
3 bool success
4 string message
```

21.7 本章练习

1. 实现巡逻行为。
 2. 实现追踪行为。
 3. 实现逃跑行为。
 4. 通过 service 切换行为。
 5. 设计一个“猫捉老鼠”小游戏。
-

第 22 章 任务队列：让小乌龟连续执行多个目标

22.1 一次只执行一个目标还不够

本章任务：

给乌龟一串目标点，让它按顺序执行。

22.2 任务的数据结构

设计：

Plain Text

```
1 task_id
2 target_x
3 target_y
4 timeout
5 status
```


22.3 任务队列管理

功能：

- 添加任务；
- 删除任务；
- 查看当前任务；
- 清空任务；
- 暂停任务；
- 恢复任务。

22.4 与 Action 结合

每个目标点作为一个 Action goal 执行。

22.5 任务失败处理

情况：

- 目标超出边界；
- 执行超时；
- 中途取消；
- 参数异常。

22.6 本章练习

1. 实现目标点队列。
2. 实现任务添加服务。
3. 实现任务取消服务。
4. 实现任务执行反馈。
5. 写一个五点巡检任务。

第八篇 小乌龟系统综合项目

第 23 章 综合项目一：自动绘图机器人

23.1 项目目标

让小乌龟根据任务参数自动绘制图形。

支持图形：

- 直线；
- 正方形；
- 正三角形；
- 正多边形；
- 圆；
- 螺旋线；
- 五角星。

23.2 系统结构

节点：

- shape_task_server；
- turtle_controller；
- pose_monitor；
- trajectory_recorder；
- debug_visualizer。

23.3 图形任务设计

每个图形任务包含：

- 图形类型；
- 起始点；
- 尺寸；
- 速度；
- 是否清屏；
- 是否记录轨迹。

23.4 正方形绘制

先实现最简单图形。

23.5 多边形绘制

将正方形推广到 N 边形。

23.6 圆形与螺旋线

通过速度控制实现连续曲线。

23.7 项目验收要求

学生需要提交：

- 源代码；
 - launch 文件；
 - 参数文件；
 - 运行截图；
 - rosbag；
 - 实验报告。
-

第 24 章 综合项目二：多乌龟追逃系统

24.1 项目目标

构建一个二维多机器人追逃系统。

角色：

- 追踪者；
- 逃跑者；
- 观察者；
- 裁判节点。

24.2 系统规则

示例规则：

- 追踪者靠近逃跑者一定距离则抓捕成功；
- 逃跑者到达安全区则逃脱成功；

- 超过时间则任务失败；
- 碰到边界扣分。

24.3 系统结构

节点：

- chaser_controller；
- runner_controller；
- referee_node；
- score_node；
- visualization_node。

24.4 追踪策略

策略：

- 直接追踪；
- 预测追踪；
- 限速追踪。

24.5 逃跑策略

策略：

- 远离追踪者；
- 走向安全区；
- 随机扰动；
- 避免靠墙。

24.6 评分系统

指标：

- 抓捕时间；
- 逃脱距离；
- 路径长度；
- 控制平滑性；

- 是否越界。

24.7 项目验收要求

学生需要展示：

- 多节点通信结构；
 - 追逃过程；
 - 参数调试过程；
 - 数据记录；
 - 策略分析。
-

第 25 章 综合项目三：小乌龟巡检系统

25.1 项目目标

模拟一个移动机器人巡检任务。

乌龟需要：

- 从起点出发；
- 依次到达多个巡检点；
- 在每个点停留；
- 发布巡检状态；
- 处理异常；
- 最后返回起点。

25.2 巡检点配置

使用 YAML 文件配置巡检点：

Plain Text

```
1 waypoints:
2   - name: A
3     x: 2.0
4     y: 2.0
5     stay_time: 2.0
6   - name: B
7     x: 8.0
8     y: 2.0
9     stay_time: 2.0
10  - name: C
11    x: 8.0
12    y: 8.0
13    stay_time: 2.0
```

25.3 巡检状态发布

状态包括：

- 当前巡检点；
- 当前任务状态；
- 已完成数量；
- 是否异常；
- 剩余任务数。

25.4 巡检任务 Action 化

将整个巡检过程设计成 Action。

25.5 异常处理

情况：

- 目标点非法；
- 到达超时；
- 用户取消任务；
- 参数缺失；

- 乌龟靠近边界。

25.6 项目验收要求

学生需要提交：

- 系统结构图；
 - 节点说明；
 - 话题说明；
 - service/action 说明；
 - 参数文件；
 - rosbag 数据；
 - 运行演示；
 - 项目报告。
-

第九篇 从小乌龟走向真实机器人

第 26 章 turtlesim 与真实移动机器人的对应关系

26.1 turtlesim 学到的东西没有浪费

对应关系：

Plain Text

1 turtlesim_node	-> 真实机器人底盘驱动节点
2 /turtle1/cmd_vel	-> /cmd_vel
3 /turtle1/pose	-> /odom 或定位结果
4 turtlesim/Pose	-> nav_msgs/Odometry
5 二维窗口边界	-> 实际环境边界
6 小乌龟轨迹	-> 机器人运动轨迹

26.2 从乌龟速度到机器人速度

讲解：

- linear.x;
- angular.z;
- 差速底盘；
- 阿克曼底盘；
- 全向底盘。

26.3 从乌龟位置到里程计

讲解：

- 乌龟位置是理想状态；
- 真实机器人位置来自传感器估计；
- 里程计会漂移；
- 定位系统会修正漂移。

26.4 从二维世界到真实传感器

对应：

- 乌龟边界检测；
- 激光雷达；
- 摄像头；
- IMU；
- 里程计。

26.5 本章练习

1. 比较 `/turtle1/cmd_vel` 和真实机器人 `/cmd_vel`。
 2. 思考 turtlesim 中缺少哪些真实机器人问题。
 3. 画出 turtlesim 系统到移动机器人系统的迁移图。
 4. 将点到点控制思想迁移到真实底盘控制。
 5. 总结 turtlesim 教学的优势和局限。
-

第 27 章 ROS1 系统设计方法总结

27.1 一个 ROS 系统应该怎样拆分节点

原则：

- 一个节点只做一类事情；
- 输入输出清楚；
- 可以单独测试；
- 参数可配置；
- 日志可观察。

27.2 Topic、Service、Action 怎样选择

总结：

Plain Text

- | | |
|----------|--------------|
| 1 连续数据流 | -> Topic |
| 2 一次请求响应 | -> Service |
| 3 长时间任务 | -> Action |
| 4 配置项 | -> Parameter |
| 5 坐标关系 | -> TF |
| 6 实验记录 | -> rosbag |
| 7 系统启动 | -> launch |

27.3 一个好 package 的样子

应该包含：

- scripts;
- launch;
- config;
- msg;
- srv;
- action;
- README;

- test;
- bag 示例数据。

27.4 一个好实验的样子

应该做到：

- 可启动；
- 可观察；
- 可记录；
- 可复现；
- 可调参；
- 可解释；
- 可扩展。

27.5 本章练习

1. 重新整理本书所有代码。
2. 为每个 package 写 README。
3. 为综合项目画系统结构图。
4. 写一份 ROS 系统设计总结。
5. 说明自己对 ROS 的理解发生了什么变化。

第 28 章 从 ROS1 到 ROS2：理解迁移，而不是死记差异

28.1 为什么需要了解 ROS2

ROS1 曾经是机器人教学和研究中的重要工具。

但新的机器人项目越来越多地转向 ROS2。

本章不是完整 ROS2 教程，而是帮助读者理解：

在 turtlesim 中学到的 ROS 思想，怎样迁移到 ROS2。

28.2 从 roscore 到 DDS

比较：

- ROS1 需要 roscore；
- ROS2 不再依赖 roscore；
- ROS2 使用 DDS 完成节点发现和通信。

28.3 从 catkin 到 colcon

比较：

- catkin_make；
- catkin build；
- colcon build。

28.4 从 rospy 到 rclpy

比较：

- 初始化节点；
- 创建 publisher；
- 创建 subscriber；
- 参数；
- spin；
- timer。

28.5 QoS：ROS2 中必须理解的新概念

解释：

- 可靠性；
- 历史深度；
- 持久性；
- 实时性；
- 为什么传感器数据和控制命令需要不同通信策略。

28.6 turtlesim 在 ROS2 中的延续

任务：

- 启动 ROS2 turtlesim;
- 发布速度命令;
- 订阅 pose;
- 调用 spawn 服务;
- 对比 ROS1 和 ROS2 命令差异。

28.7 本章练习

1. 用 ROS2 启动 turtlesim。
 2. 对比 ROS1 和 ROS2 的 topic 命令。
 3. 对比 ROS1 和 ROS2 的 service 命令。
 4. 将一个 ROS1 小乌龟控制节点改写为 ROS2 版本。
 5. 总结 ROS1 与 ROS2 在系统思想上的连续性。
-

附录 A Ubuntu 与 Linux 命令基础

A.1 目录操作

- pwd
- ls
- cd
- mkdir
- rm
- cp
- mv

A.2 文件权限

- chmod
- chown
- 可执行脚本权限

A.3 终端与环境变量

- source
- echo
- bashrc
- 环境变量查看

A.4 软件安装

- apt update
 - apt install
 - apt search
-

附录 B ROS1 常用命令速查

B.1 节点命令

- rosnodet list
- rosnodet info
- rosnodet kill

B.2 话题命令

- rostopic list
- rostopic echo
- rostopic info
- rostopic hz
- rostopic pub

B.3 消息命令

- rosmst list
- rosmst show

B.4 服务命令

- rosservice list
- rosservice info
- rosservice call

B.5 参数命令

- rosparam list
- rosparam get
- rosparam set
- rosparam load
- rosparam dump

B.6 bag 命令

- rosbag record
 - rosbag play
 - rosbag info
-

附录 C Python ROS 编程速查

C.1 节点

- rospy.init_node
- rospy.spin
- rospy.Rate

C.2 Publisher

- rospy.Publisher
- publish

C.3 Subscriber

- `rospy.Subscriber`
- `callback`

C.4 Service

- `rospy.Service`
- `rospy.ServiceProxy`

C.5 Parameter

- `rospy.get_param`
- `rospy.set_param`

C.6 Time

- `rospy.Time`
- `rospy.Duration`

C.7 Log

- `rospy.loginfo`
 - `rospy.logwarn`
 - `rospy.logerr`
-

附录 D turtlesim 常用接口速查

D.1 主要节点

- `turtlesim_node`
- `turtle_teleop_key`

D.2 常用话题

- /turtle1/cmd_vel
- /turtle1/pose

D.3 常用服务

- /spawn
- /kill
- /clear
- /reset
- /turtle1/teleport_absolute
- /turtle1/teleport_relative
- /turtle1/set_pen

D.4 常用消息

- turtlesim/Pose
 - geometry_msgs/Twist
-

附录 E 课程实验安排建议

实验 1：启动 turtlesim 与观察 ROS 系统

目标：理解 roscore、node、topic。

实验 2：编写第一个 ROS 节点

目标：理解节点身份和节点命名。

实验 3：程序控制小乌龟运动

目标：掌握 publisher 和 Twist 消息。

实验 4：读取小乌龟位置

目标：掌握 subscriber 和 callback。

实验 5：点到点控制

目标：理解反馈控制思想。

实验 6：自定义消息

目标：掌握 msg 文件和消息生成流程。

实验 7：Service 控制乌龟

目标：掌握请求响应式通信。

实验 8：Action 移动任务

目标：掌握长任务、反馈和取消机制。

实验 9：参数与 launch

目标：实现可配置、一键启动的小乌龟系统。

实验 10：多乌龟追踪

目标：掌握多节点、多话题、命名空间。

实验 11：TF 与相对坐标

目标：理解坐标变换思想。

实验 12：rosviz 与数据分析

目标：掌握实验记录和复现方法。

实验 13：状态机巡逻

目标：理解机器人任务流程组织。

实验 14：多乌龟追逃系统

目标：完成多机器人协同项目。

实验 15：小乌龟巡检系统

目标：完成完整 ROS1 综合项目。

附录 F 期末项目评价标准

F.1 功能完整性

- 是否能够正常启动；
- 是否完成指定任务；
- 是否支持参数配置；
- 是否支持异常处理。

F.2 ROS 规范性

- 节点划分是否合理；
- 话题命名是否清晰；
- service/action 设计是否合适；
- launch 是否完整；
- package 结构是否规范。

F.3 可调试性

- 是否能用 rqt_graph 看清结构；
- 是否有日志输出；
- 是否能用 rostopic 检查数据；
- 是否能录制 rosbag；
- 是否能复现实验。

F.4 报告质量

- 是否有系统结构图；
- 是否说明关键代码；
- 是否展示实验结果；
- 是否分析失败案例；
- 是否提出改进方案。

F.5 创新拓展

- 是否设计了新的乌龟行为；
- 是否实现了更复杂的多乌龟协同；
- 是否加入了可视化；
- 是否加入了评分系统；
- 是否能迁移到真实机器人思想。